

AIPL 3 実装の同期

— 拡張子の統一、仕様追従、そして調べる過程で出た 6 つの不具合 —

児玉工房 / AICE · AIPL プロジェクト

2026 年 08 月 01 日 00:25

概要

AIPL には実装が 3 つある — 正である OCaml 版、Python の CLI 実装 (Py-I)、ブラウザで動く JavaScript 実装 (JS-I)。OCaml 版に加えた一連の仕様変更に残る 2 つが追いついておらず、**ガイドのサンプル 8 本が Py-I でも JS-I でも 1 本も動かない状態**になっていた。

本レポートはこれを揃えた作業の記録である。やったことは二つ。ソースの拡張子を `.abc1` から `.aipl` へ統一し (4 リポジトリ 478 本)、Py-I と JS-I を最新の言語仕様へ追従させた。結果、サンプルは 3 実装すべてで動き、出力も一致する。

ただし本レポートの主眼は作業一覧ではない。**揃える過程で 6 つの不具合が出てきたことと、それがどうやって見つかったか**である。うち 3 つは正であるはずの OCaml 版のバグだった。とくに `g4_select` の挙動差を「スケジューリングの違い」と片付けずに**どちらが正しいか**を調べたことが、トップレベル送信の順序破壊 → `select` の返信経路の欠落 → アクターを引数で渡せない、という連鎖の発見につながった。

見つけ方の型は 3 つに集約できる。**3 実装を並べて件数を比べる、機械的な置換をせず構文木で判定する、検査の指摘は件数でなく実物を開いて確かめる**。最後の型は実際に効き、reply 検査の 62 件の指摘のうち 59 件が誤検出であることを暴いた。

目次

1	三つの実装	2
1.1	着手前の状態	2
2	拡張子の統一	3
3	仕様の追従	3
3.1	Py-I は書式が違っていただけの箇所があった	3
4	+ から ++ への移行 — 正規表現を使わなかった理由	4
4.1	構文木は文字列リテラルの中を見ない	4
4.2	検証	4
5	見つかった 6 つの不具合	4
5.1	(1) トップレベルの <code>send</code> がプログラム順を破っていた	5
5.2	(2)(3) <code>select</code> の <code>case</code> 本体の <code>reply</code> が呼び出し元へ戻らない	5
5.3	(4) 期限検査がトップレベルを見ていなかった — 両実装で	6
5.4	(5) JS-I の <code>wait</code> がブロックしない	6

5.5	(6) アクターをメッセージの引数として渡せない	6
6	見つけ方の型	7
6.1	3つ並べて件数を比べる	7
6.2	機械的な置換をせず構文木で判定する	7
6.3	検査の指摘は件数でなく実物を開いて確かめる	7
6.4	おまけ — 警告が出力経路に載っているか	8
7	退行させた一度と、その気づき方	8
8	最終状態	8
8.1	ガイドのサンプル	8
8.2	g9: アクターを引数で渡す	8
8.3	残っている差	9
9	コミット一覧	9
10	まとめ	10

1 三つの実装

場所	備考
OCaml 版 ~/aios/abclcp	正。10.8k 行。REPL から load / compile
Py-I ~/test-bed/aios-claude/src/python-aip1	CLI。15.7k 行。Lark 文法(grammar.lark)
JS-I ~/projects/drone-hil/abcl	ブラウザ実装。5.2k 行。jison 文法を parser.js へ生成

JS-I がブラウザ実装であることは以降の随所に効いてくる。デモは HTML ページで、index.html が parser/parser.js を <script> で読み込み、interpreter.js を ES モジュールとして import する。検証のために node から parser.yy に ast.js を束ねて Interpreter を回すヘッドレスの足場を組んだ。

1.1 着手前の状態

ガイドのサンプル (docs/samples/guide/g1..g8) を各実装に流したところ、Py-I も JS-I も 8 本すべて失敗した。Py-I の実際のエラーはこうである。

```
g8: [parse error] No terminal matches 'f' ... var flag = false;
g7: [parse error] Unexpected token Token('COLON', ':')
g1: [parse error] Unexpected token Token('PLUS', '+')
```

真偽値リテラル、戻り値型注釈 : T、文字列連結 ++ のいずれもが無い。JS-I も grammar.jison に ++ :: !{ の規則そのものが無かった。

2 拡張子の統一

言語名は AIPL なのにソースの拡張子が `.abcl` のままだった。前身の ABCL に由来する名残で、3 実装をまたいで混乱の元になっていた。

リポジトリ	改名	参照追跡	コミット
<code>aio/abclcp</code>	93	50	<code>cccb29d</code>
<code>test-bed/aio-claude</code>	380	443	<code>2ac81b9</code>
<code>projects/drone-hil</code>	3	6	<code>b78ec34</code>
<code>aio_line_simulator</code>	2	10	<code>7856575</code>
合計	478	509	

履歴を保つため `mv` ではなく `git mv` を使った。参照は `.sh`・`.bat`・`.md`・`.py`・`.json`・`.tex`・`.js` など広範に及んだが、拡張子を決め打ちしているコードは `aio_dashboard.py` の 2 箇所 (`endswith(".abcl")`) だけだった。処理系のパーサ・字句解析器・REPL は拡張子を見ていない。

検証は「型検査の結果が改名前と 1 件ずつ一致するか」で行い、**93 本中 93 本が完全一致**した。うち 27 本は `PARSE_ERROR` だが、これは改名前から同じ旧構文の残骸であり本件の退行ではない。

3 仕様の追跡

OCaml 版に入っていて Py-I / JS-I に無かったものを移した。

	OCaml	Py-I	JS-I
<code>++</code> (文字列連結)	既	○追加	○追加
真偽値リテラル	既	○追加	○追加
優先順位の分離	既	○追加	○追加
<code>timeout ... else</code> (期限)	既	○追加	○追加
戻り値型注釈 <code>: T</code>	既	○別名追加	○追加
効果注釈 <code>!{...}</code>	既	既	○追加
<code>+</code> を数値専用に	既	○移行	○追加
引数の型注釈 <code>method m(x: T)</code>	○追加	既	○追加
<code>reply</code> 線形性の検査	既	○追加	○追加
期限の警告	既	○追加	○追加

優先順位は OCaml 版に合わせて分離した。低い順に

等値 < 比較 < `++` < 加減 < 乗除 < 単項マイナス

であり、二項演算子はすべて左結合である (等値も含む)。これは実測で確かめた `-- 1 == 2 == false` は `((1 == 2) == false)` と読まれて `true` になり、`-- 3` は 3 になる。

3.1 Py-I は書式が違っていただけの箇所があった

Py-I の `grammar.lark` には既に `return_anno` と `effect_anno` があった。無かったのではなく書式が違っていた。

	OCaml 版	Py-I
戻り値型注釈	<code>method m(x) : T</code>	<code>method m(x) -> T</code>

両方受けるようにした。注釈が黙って捨てられていないことは、パースした AST を直接覗いて確かめた (`ret='int' eff=['log']`)。「通ったが無視されていた」は AIPL の道具立てで何度も踏んだ罠である。

4 + から ++ への移行 — 正規表現を使わなかった理由

OCaml 版では + は数値専用で、文字列連結は ++ に分離済みである（オーバーロードの曖昧さを潰すため）。同じことを Py-I でやったところ、テストが 15 通過/5 失敗 → 6 通過/14 失敗 に悪化した。調べると既存の .aipl 資産 437 本のうち 263 本が + で文字列を連結していた。

移行が必要だが、正規表現で置換してはいけない。+ は左結合で ++ より強く結合するので、連鎖の + を 1 つだけ替えると意味が変わる。

```
"a" + b + c      // ((("a" + b) + c) ---- 全体が文字列連結
"a" ++ b + c     // "a" ++ (b + c) ---- b と c が数値なら加算に化ける
```

そこで構文木で判定した。Lark を `propagate_positions=True` で生の木に落とし、`add_expr` の連鎖ごとに「オペランドのどれかが文字列か」を見て、該当する連鎖の + トークンの位置をすべて集め、末尾から置換する (+ 1 文字 → ++ 2 文字で以降の位置がずれるため)。

238 本 / 5448 箇所を書き換え

4.1 構文木は文字列リテラルの中を見ない

これで済んだと思ったらテストが 2 本落ちた (`_test_dynamic`, `_test_methodpatch`)。原因は、実行時にコンパイルする AIPL ソースを文字列として持つファイルがあったこと。

```
"    print(\"    [Counter] tick -> n=\" + n);\n\" ++
```

外側の連結は ++ に直っているのに、文字列の中の + は構文木からは見えないので残る。\" を含む行を別途処理して解消した (8 本 11 行)。

4.2 検証

移行後、437 本中 413 本がパースできる。失敗 24 本は移行前と同数・同一である (`Generics.aipl` の型引数など、元から通らない旧構文)。テストは 15 通過/5 失敗で基準と完全一致し、新規失敗はゼロだった。

5 見つかった 6 つの不具合

ここからが本題である。揃える作業そのものより、揃えようとして初めて見えた不具合のほうが多く、しかもその半分は正であるはずの OCaml 版にあった。

5.1 (1) トップレベルの send がプログラム順を破っていた

g4_select の出力が 3 実装で食い違っていた。

OCaml 版	direct:1 / waiting / timed out
Py-I, JS-I	waiting / got 1

これを「スケジューリングの違い」で片付けず、どちらが正しいのかを最小の例で測った。

```
class A {
  method first() : unit !{log} { print("1st: first"); }
  method second() : int !{log} { print("2nd: second"); reply(7); }
}
var a = new A();
send a.first();
print("now returned " ++ (now a.second() timeout 1000 else 0));
```

OCaml 版	2nd: second / now returned 7 / 1st: first
Py-I, JS-I	1st: first / 2nd: second / now returned 7

OCaml 版だけが FIFO を破っていた。5 回ずつ走らせて、どちらも決定的であることを確かめている（競合ではない）。

原因はメールボックスではなかった。アクターの queue は単一の Queue.t で、actor_loop も Queue.pop の厳密な FIFO、send_message も mutex 下で即 push する。つまり配送は正しい。ずれていたのは repl_thread.ml の compile で、**トップレベルの Send だけを pending_global_sends に積んで全文の実行後にまとめて送っていたのに**、CallStmt (print(now ...) など) はその場で実行していたことだった。

後回しは前方参照のために入っていたと思われるが、対象 93 本を調べたところ send が var ... = new ... より前に来るファイルは 0 本だった。それでも壊さないよう、**宛先が既に生成されていればその場で送り、まだ無い場合だけ後回しにする条件付きに変えた** (df66dca)。

これで g4_select は waiting / got 1 / via select:1 となり、ガイドの説明「select で job を待つ。case 本体の reply は job への返信になる」と一致した。

5.2 (2)(3) select の case 本体の reply が呼び出し元へ戻らない

(1) を直したことで g4 が**実際に** select 経路を通るようになり、そこで Py-I と JS-I が破綻していることが分かった。

Py-I	値が返らず [REPLY] via select:1 とログするだけ
JS-I	now deadlock: slot rs-1 not fulfilled で失敗

原因は両方とも同じで、case 本体を実行するときに**受け取ったメッセージの返信先を据えていなかった**こと。親フレーム (select を書いたメソッド自身) の返信先を辿ってしまっていた。Py-I は drained[i] の reply_future を捨てており、JS-I は actor.__currentSlotId を差し替えていなかった。それぞれ据えるよう直した (e7b6295, c2929f5)。

5.3 (4) 期限検査がトップレベルを見ていなかった — 両実装で

JS-I に期限検査を移し、OCaml 版と件数を並べて比べたところ合わなかった。

期限警告の件数	OCaml	Py-I	JS-I
g3_actors	3	1	1
g6_effects	3	1	1

原因はクラスのメソッドしか走査しておらず、`print(now front.place(3));` のようなトップレベルの `now` を見ていなかったこと。JS-I を直して比べ直したら、同じ穴が Py-I にもあった (`GlobalStmt` を通っていないかった)。両方直して全項目が一致した (22c0569, 9c7696c)。

1 実装だけ見ていたら両方見逃していた。

5.4 (5) JS-I の `wait` がブロックしない

`g7_deadline` だけが最後まで揃わなかった。

OCaml 版, Py-I	<code>slow(timed out) = 0</code>
JS-I	<code>slow(timed out) = 42</code> (期限が発火しない)

OCaml 版と Py-I の `wait(ms)` はメソッド実行中にブロックするので、その後の `reply` も遅れる。JS-I の `wait` は `actor.__nextDelay` を立てて次のメッセージ配送を遅らせるだけで、実行中の処理はブロックしない。ブラウザで同期ブロックすると画面が固まるための設計であって手抜きではない。

意味を変えるのは実行基盤に踏み込むので、先に影響を数えた — `drone-hil` 全体で `wait` と `reply` を同居させているメソッドは 0 個だった (`wait` はデモのアニメーションの間合いにしか使われていない)。そこでブロックの代わりに「この先の `reply` を `ms` だけ遅らせる」形で近似した (4b6f2b5)。

- `wait`: `actor.__replyDelayMs` に加算
- `Reply`: 遅延があれば即 `fulfill` せず `{slotId, value, notBefore}` を積む
- `_drainOneStep`: 毎回フラッシュし、待ちが残っていれば「まだ進む余地がある」として `true` を返す (`false` だと `drainUntilSlot` が `deadlock` 例外を投げる)

これで 5 回とも `slow(timed out) = 0` で安定した。

5.5 (6) アクターをメッセージの引数として渡せない

引数の型注釈を入れたあとに見つかった。

```
var d = new D();
send p.take(d);      // take が呼ばれない。エラーも出ない
```

アクター同士が相手を教え合う基本的な書き方が封じられていた。注釈の有無に関わらず再現するので、注釈とは無関係の既存の不具合である。原因は 3 つ重なっていた。

■(6a) 値を式へ戻すときアクターを文字列に潰していた。メッセージ引数は値へ評価してから `expr_of_value` で式に戻して送る。その `expr_of_value` が

```
| VActor (n,_) -> String ("<actor:" ^ n ^ ">")
```

としており、受け手に届くのはアクターではなく文字列だった。真偽値も `String "true"` に潰れていた (`Bool` の式型ができたのに古いまま)。 `expr_desc` に `ActorRef` を追加し、 `VActor → ActorRef`、 `VBool → Bool` とした。

■(6b) トップレベルの `send` が引数を評価していなかった。生の式のまま `send_message` に渡していたので、受け手の環境で `Var "d"` が解決できず黙って落ちていた (`actor_loop` が例外を握って `web` イベントに積むだけなので無反応に見える)。

■(6c) `VActor` が実体名を持っていなかった。 `VActor` の第 1 要素はクラス名なのだが、 `resolve_actor_from_term` はそれを実体名として読んでおり、コード内で一貫していない。実体名とクラス名が一致する場合と、変数名で引ける場合にだけ動いていた — 引数で受け取ったアクターはそのどちらでもない。

`actor_value ~name ~cls` を用意し、第 1 要素はクラス名のまま実体名を表の `"__actor_name"` に入れ、 `actual_local_target` と `resolve_actor_from_term` が「実体名 → 第 1 要素 → 変数名」の順で引くようにした。第 1 要素の意味は変えていないので `typeof` 等は無傷である (`cbaa7bc`)。

6 見つけ方の型

同じ発見が 3 つの型に集約できる。他の多実装プロジェクトにもそのまま効くと思う。

6.1 3 つ並べて件数を比べる

§5.4 がそれである。1 実装を見て「出た」で満足すると過小報告に気づかない。同じ入力に対する件数を並べると、単体では正しく見える実装の穴が出る。

6.2 機械的な置換をせず構文木で判定する

§4 がそれである。演算子の優先順位が絡む書き換えは、正規表現では意味を変えずに行えない。構文木で連鎖を判定し、バイト位置で末尾から置換する。ただし構文木は文字列リテラルの中を見ないので、実行時コンパイルを持つ言語ではそこが漏れる。

6.3 検査の指摘は件数でなく実物を開いて確かめる

`reply` の線形性検査を入れたとき、全 437 本で **28 本 62 件** の指摘が出た。件数だけ見れば「よく効いている」と読めるが、開いてみると全て

```
method add(x: int, y: int) -> int { return x + y; }
```

の形だった。Py-I のメソッドは OCaml 版と違い `return` でも返せる。 `return` を勘定に入れたら **62 件が 3 件に落ちた**。その前段でも、 `function` (`reply` ではなく `return` で返す) に検査をかけて `_test_linear` を落としている。

残った 3 件は本物だった。

```
method read() {
  if (kind == "Formula") { var v = eval_one(...); reply(v); }
  reply(val);           // if に else も脱出も無いので落ちてくる
}
```

kind == "Formula" のとき 2 回 reply する。Future.set が冪等なので実害は出ていないが（先勝ちで v が返る）、2 つ目は死んだコードである。

指摘は「出たら本物」でないと使われなくなる。件数で納得すると、誤検出だらけの検査を仕込むことになる。

6.4 おまけ — 警告が出力経路に載っているか

Py-I に期限の警告を足したのに、--type-check にも type_check() にも現れなかった。module 直下の check() が issues しか返していなかったためである。検査を足しただけで見えないままなら、無いのと同じである。

7 退行させた一度と、その気づき方

引数の型注釈を JS-I に入れたとき、g4_select が now deadlock: slot rs-1 not fulfilled に戻った。params を使う文法規則が method_decl と select_case の 2 つあり、片方だけ直したためである。params を {name, ty} の並びに変えた結果、case 本体の束縛 localEnv[p] がオブジェクトをキーにしてしまい、引数が見えなくなっていた。

ガイドの 8 本を毎回流していなければ気づけなかった。検証を「変更に関係ありそうな範囲」に絞らないことが効いた場面である。

8 最終状態

8.1 ガイドのサンプル

	OCaml	Py-I	JS-I	一致
g1_hello	○	○	○	一致
g2_now_future	○	○	○	一致
g3_actors	○	○	○	一致
g4_select	○	○	○	一致
g6_effects	○	○	○	一致
g7_deadline	○	○	○	一致
g8_bool_equality	○	○	○	一致
g9_actor_arg	○	○	○	一致

(g5_web は Web サーバを立てるので出力比較の対象外) 着手前は Py-I・JS-I とも 0/8 だった。

8.2 g9: アクターを引数で渡す

本作業で追加したサンプルである。§5.5 の修正がそのまま効く。

```
// g9: アクターをメッセージの引数として渡す
```



```
//
// `method m(x: D)` と引数に型注釈を書くと、受け取ったアクターに
// そのまま send できる。注釈が無いと引数の型が決まらないので
// `send target is not actor` になる。
//
// 長らく動かなかった書き方で、原因は3つ重なっていた:
// 1. expr_of_value が VActor を "<actor:d>" という文字列に潰していた
// 2. トップレベルの send が引数を評価せず生の式のまま送っていた
// 3. VActor が実体名を持たず、宛先を復元できなかった
class D { method ping() : unit !{log} { print("pong"); } }
class P {
  method viaAnno(x: D) : unit !{log} {
    print("got actor");
    send x.ping();
    print("after send");
  }
}
var d = new D();
var p = new P();
send p.viaAnno(d);
```

3 実装とも got actor / after send / pong

なお最初は `typeof(x)` を使って書いていたが、`typeof` は JS-I に無い (Unknown function: `typeof`) ため 3 実装共通で動かず、外した。サンプルは 3 実装で動くことに意味がある。

8.3 残っている差

項目	内容
typeof の書式	OCaml: <code>actor(D) { ping : () -> unit }</code> , Py-I: <code>actor(D, methods=[ping])</code> 。JS-I には存在しない
メソッド内の <code>return</code>	Py-I は受けるが JS-I は受けない。bench/*.aipl 3 本がこれで止まっている
アクター型の区別	OCaml の <code>unify</code> は <code>TActor</code> 同士を無条件に成功させる。 <code>actor[A]</code> と <code>actor[B]</code> が単一化できてしまう
出力のフラッシュ	OCaml の REPL がスクリプト終了時に最後の 1 行を取りこぼすことがある。実装差ではなく出力の揺らぎ

9 コミット一覧

リポジトリ	コミット	内容
abclcp	cccb29d	拡張子を .aipl へ (93 本)
abclcp	df66dca	トップレベル <code>send</code> の順序 (§5.1)
abclcp	068bdcd	引数の型注釈 <code>method m(x: T)</code>
abclcp	cbaa7bc	アクターを引数で渡せるように (§5.5)
abclcp	47e3fa1	g9 から <code>typeof</code> を外す
aiois-claude	2ac81b9	拡張子を .aipl へ (380 本)

リポジトリ	コミット	内容
aio-s-claude	ce60fbd	Py-I を最新仕様へ追従
aio-s-claude	9ccc052	+ を数値専用、241 本を移行
aio-s-claude	85b6f2d	reply 線形性と期限の検査
aio-s-claude	9c7696c	期限検査がトップレベルを見ていなかった件
aio-s-claude	e7b6295	select の返信経路
drone-hil	b78ec34	拡張子を .aipl へ (3 本)
drone-hil	db7f5da	JS-I を最新仕様へ追従
drone-hil	bee8408	+ を数値専用
drone-hil	22c0569	reply 線形性と期限の検査
drone-hil	c2929f5	select の返信経路
drone-hil	4b6f2b5	wait が reply を遅らせる
drone-hil	601dd38	引数の型注釈
aipl_line_simulator	7856575	拡張子を .aipl へ (2 本)

abclcp は push 済み。他の 3 リポジトリは作業中の feature ブランチ上にあるためコミットのみで push は保留している。

10 まとめ

1. 拡張子を .aipl に統一した (4 リポジトリ 478 本、参照 509 ファイル)。型検査の結果が改名前と 93/93 で一致することを確認している。
2. Py-I と JS-I を最新の言語仕様へ追従させた。ガイドのサンプルは 0/8 → 8/8、しかも 3 実装で出力が一致する。
3. + → ++ の移行は構文木で連鎖を判定して行った (238 本 5448 箇所)。正規表現では優先順位のせいで意味が変わる。
4. 揃える過程で 6 つの不具合が出た。うち 3 つは正であるはずの OCaml 版のものだった — トップレベル送信の順序破壊、アクターを引数で渡せない件 (原因 3 つ)、そして expr_of_value が真偽値まで文字列に潰していたこと。
5. g4_select の差を「スケジューリングの違い」で片付けずどちらが正しいかを測ったことが、連鎖的な発見の起点になった。片付けていれば 3 つとも見つかっていない。
6. 見つけ方は 3 つの型に集約できる — 3 実装を並べて件数を比べる、機械的な置換をせず構文木で判定する、検査の指摘は件数でなく実物を開いて確かめる。3 つ目は 62 件の指摘のうち 59 件が誤検出であることを暴いた。

参考

- docs/aipl_guide_ja.pdf — AIPL ユーザーズガイド (26 ページ)。サンプル 8 本をソース全文と実行結果つきで解説。
- docs/aipl_soundness2_ja.pdf — 型健全性と型安全性の第 2 版。期限を必須にすると進行定理が二択になる。

- docs/aip1_reply_inference_ja.pdf — reply からの戻り値型推論と注釈、線形性。
- docs/samples/guide/g1..g9 — 本レポートで参照したサンプル。